

AgroSearch — Motor de Busca Inteligente

UNIPÊ · Tópicos Avançados – Recuperação de Informação / PLN · Laboratório Prático 04 (Desafio Integrador) · Prof. Me. Ricardo Roberto de Lima

Equipe: Max Cavalcante · [Nome do integrante 2] · [Nome do integrante 3]

1. Visão geral

O AgroSearch é um protótipo de busca textual para os manuais da AgroTech Solutions. Uma única aplicação Streamlit (`agrosearch.py`) integra os três pilares da disciplina: pré-processamento, índice invertido e ranqueamento TF-IDF, com bônus de similaridade de cosseno. Conforme a restrição do desafio, **nenhuma biblioteca de alto nível** (`scikit-learn`, `TfidfVectorizer`, `NLTK`) foi usada: tudo foi escrito com *math*, *re* e *unicodedata*; o `pandas` serve apenas para exibir tabelas. A base é a de 5 documentos sugerida, editável na barra lateral. Execução: `streamlit run agrosearch.py`.

2. Decisões de implementação

Fase 1 — Pipeline. (i) *Tokenização* por expressão regular; (ii) *normalização* em minúsculas e sem acentos (decomposição NFD, descartando marcas de acento); (iii) *stopwords*, com lista em português já normalizada, pois a remoção ocorre depois da normalização (“é” vira “e”); (iv) *stemming* por regras de sufixo inspiradas no RSLP: redução de plural (`lagartas`→`lagarta`), sufixos como `-ação`, `-mento`, `-idade`, `-adora`, `-ico`, `-osa` e remoção da vogal final. Dois *checkboxes* ligam/desligam *stopwords* e *stemming*, e a aba mostra cada etapa e o tamanho do vocabulário em tempo real. A **consulta passa exatamente pelo mesmo pipeline** dos documentos; sem isso os termos não casariam com o índice.

Fase 2 — Índice invertido. Dicionário *termo* → *lista de IDs* construído sobre os tokens finais, sem repetições, exibido em tabela (com DF) e em JSON. O DF de cada termo é o tamanho da sua lista, e a consulta só precisa olhar as listas dos seus termos em vez de varrer todo o texto.

Fase 3 — TF-IDF. $TF(t,d) = f(t,d) / |d|$ ($|d|$ = tokens do documento após o pré-processamento); $IDF(t) = \ln(N / DF(t))$, com $IDF = 0$ se o termo não existe na coleção; $TF-IDF = TF \cdot IDF$. Para consultas com várias palavras, o score é o **TF-IDF acumulado** (soma sobre os termos da consulta). A tabela é ordenada do maior para o menor e o vencedor é destacado. **Bônus:** similaridade de cosseno entre o vetor TF-IDF da consulta e o de cada documento (a norma do documento usa todo o vocabulário dele), com opção de ordenar o ranking por esse critério.

3. Resultados

Vocabulário (base padrão): 51 tokens únicos brutos → 47 após normalização → **33** após remoção de stopwords (o stemming altera os radicais; aqui não muda a contagem porque cada palavra já aparece em uma só forma, e por isso o ranking a seguir é o mesmo com e sem stemming). Consulta **“lagartas na soja”** → tokens `['lagart', 'soj']`. $IDF: lagart = 0.9163$ ($DF=2$); $soj = 0.5108$ ($DF=3$).

Pos.	Documento	TF-IDF acumulado	Cosseno	Trecho
1	Doc 2	0.2039	0.2799	O controle biológico de lagartas na soja pode ser feito com a...
2	Doc 4	0.1784	0.2572	Lagartas desfolhadoras causam grande prejuízo na cultura da S...
3	Doc 1	0.0639	0.0610	A soja requer irrigação constante durante o período de floraç...
4	Doc 3	0.0000	0.0000	A adubação verde com leguminosas melhora o nitrogênio no solo...
5	Doc 5	0.0000	0.0000	A irrigação por gotejamento economiza água e é ideal para o c...

O **Doc 2** vence: contém os dois termos, e por ser curto (7 tokens) o TF é alto. O Doc 4 vem em seguida, também com os dois termos, porém com 8 tokens; a mesma ordem se repete no cosseno. Já para **“controle biológico lagartas”** o Doc 2 lidera com folga (TF-IDF 0.5907; Doc 4: 0.1145), pois reúne três termos raros. **Efeito do stemming:** a consulta “lagarta” (singular) retorna **0 documentos sem stemming e 2 com stemming** (Doc 2, Doc 4), porque “lagarta” e “lagartas” passam ao mesmo radical. O cosseno ajuda em consultas longas por normalizar pelo tamanho dos vetores.

4. Divisão de tarefas

Integrante	Responsabilidades
Max Cavalcante	Fase 1: pipeline de pré-processamento, stemmer, stopwords e checkboxes; aba de pré-processamento.
[Nome do integrante 2]	Fases 2 e 3: índice invertido, cálculo de TF, IDF, TF-IDF acumulado, ranking e destaque do vencedor.
[Nome do integrante 3]	Bônus (cosseno), interface unificada, testes com consultas, revisão do código e redação deste relatório.

5. Observações e limitações

Ao reaproveitar as práticas 1 a 3, corrigimos três pontos: a prática 1 não removia pontuação (“correu.” ≠ “correu”); na prática 3 a contagem de TF comparava a consulta em minúsculas com palavras que não estavam normalizadas; e o stemmer da prática 2 cortava um caractere a menos que o sufixo (“rapidamente” → “rapida”). O stemmer é aproximado e pode agrupar ou separar palavras de forma imperfeita (ex.: “requer” → “requ”); o IDF

com $\ln(N/DF)$ zera termos presentes em todos os documentos; e, como o índice guarda só os IDs, o TF é obtido dos tokens de cada documento (uma extensão natural seria guardar as frequências no índice).